



D Data
S Structures
A Algorithms

Data Structures & Algorithms

Today's Topic: Graph Traversal



Dr. Si Thu Aung



in/drsithuaung/

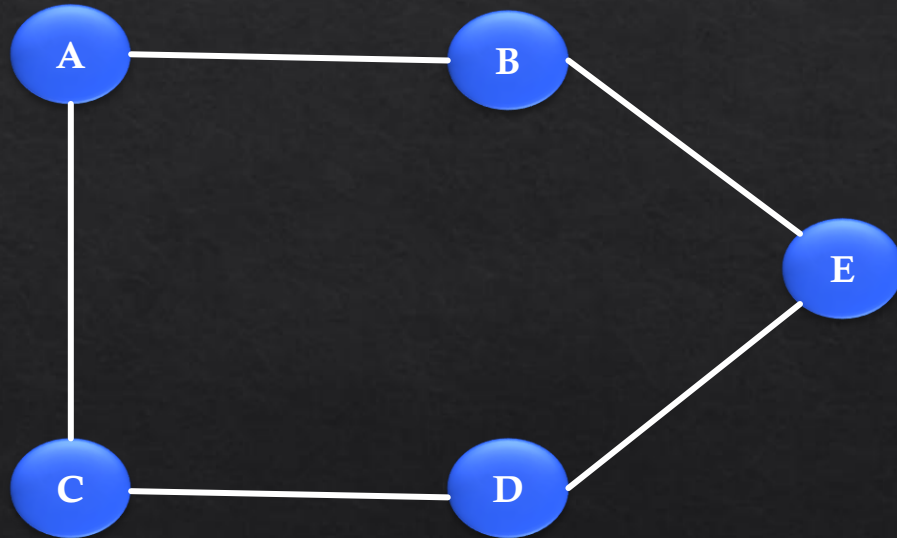


<https://sithuaung-ink.github.io/data-analyst-portfolio/>

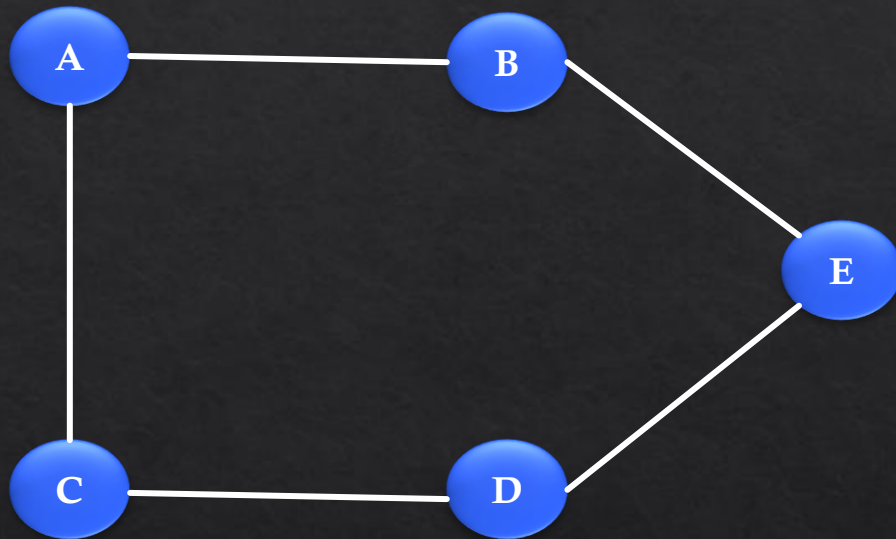
5/7/2026

Graph Traversal

- Breadth First Search – BFS
- Depth First Search – DFS



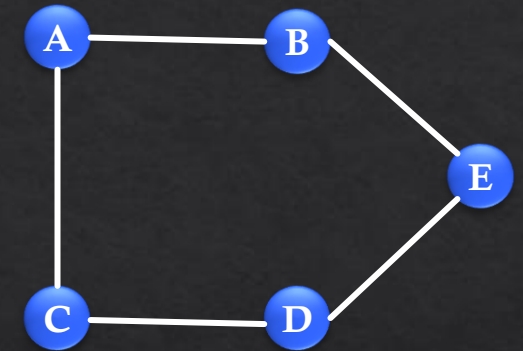
Breadth First Search – BFS



{
A: [B, C],
B: [A, E],
C: [A, D],
D: [C, E],
E: [B, D]
}

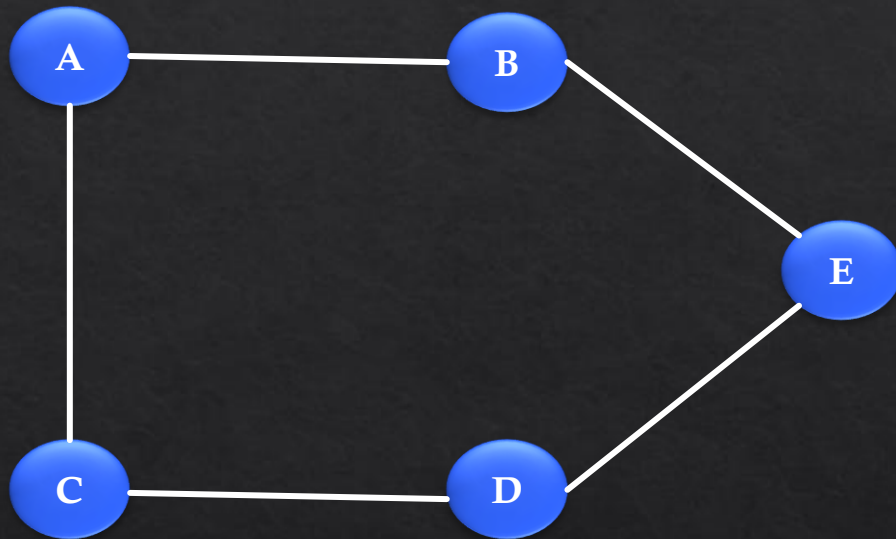
Breadth First Search – BFS

```
def bfs(self, vertex):  
    visited = set()  
    visited.add(vertex)  
    queue = [vertex]  
    while queue:  
        current_vertex = queue.pop()  
        print(current_vertex)  
        for adjacent_vertex in self.adjacency_list[current_vertex]:  
            if adjacent_vertex not in visited:  
                visited.add(adjacent_vertex)  
                queue.append(adjacent_vertex)
```



```
{  
    A: [B, C],  
    B: [A, E],  
    C: [A, D],  
    D: [C, E],  
    E: [B, D]  
}
```

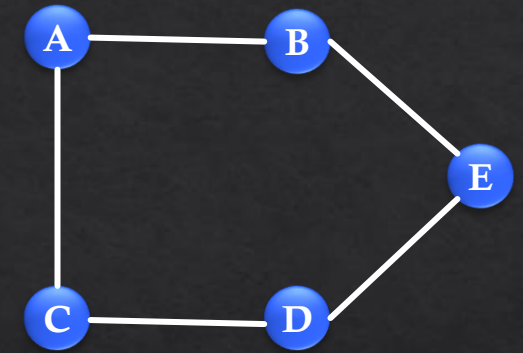
Depth First Search – DFS



{
A: [B, C],
B: [A, E],
C: [A, D],
D: [C, E],
E: [B, D]
}

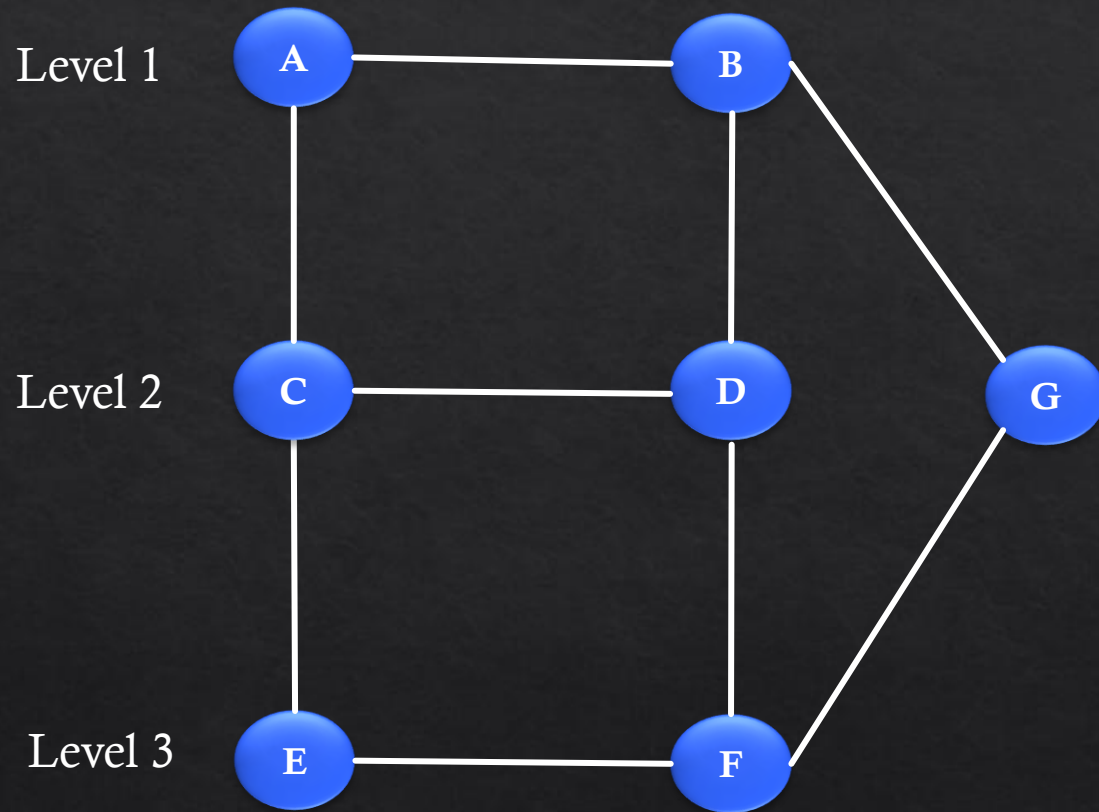
Depth First Search – DFS

```
def dfs(self, vertex):  
    visited = set()  
    stack = [vertex]  
    while stack:  
        current_vertex = stack.pop()  
        if current_vertex not in visited:  
            print(current_vertex)  
            visited.add(current_vertex)  
        for adjacent_vertex in self.adjacency_list[current_vertex]:  
            if adjacent_vertex not in visited:  
                stack.append(adjacent_vertex)
```

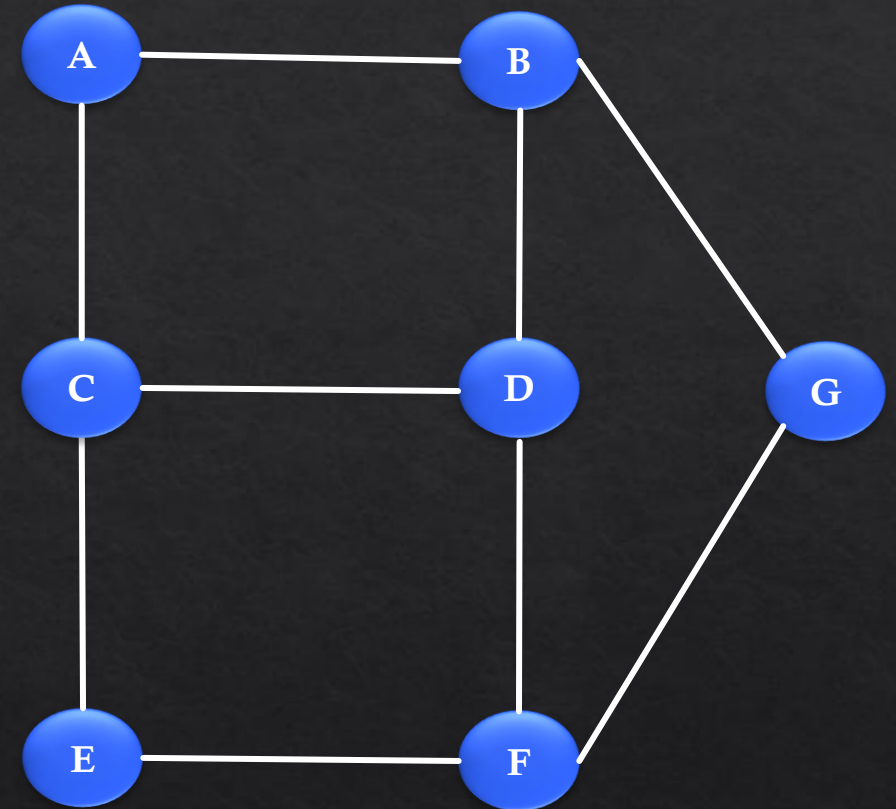


```
{  
    A: [B, C],  
    B: [A, E],  
    C: [A, D],  
    D: [C, E],  
    E: [B, D]  
}
```

BFS vs DFS



BFS



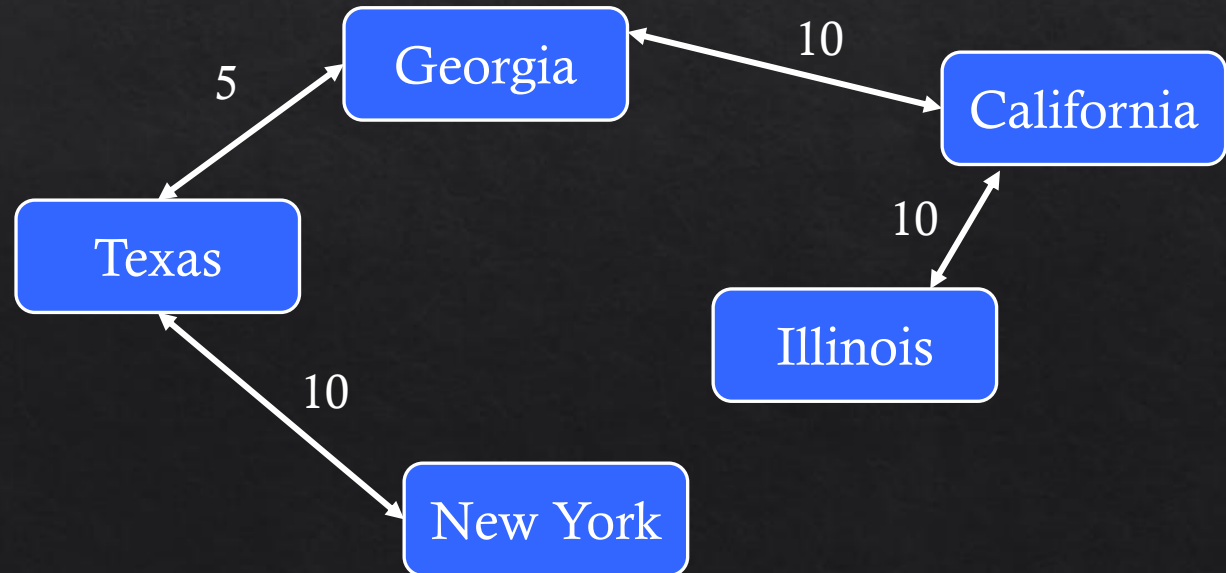
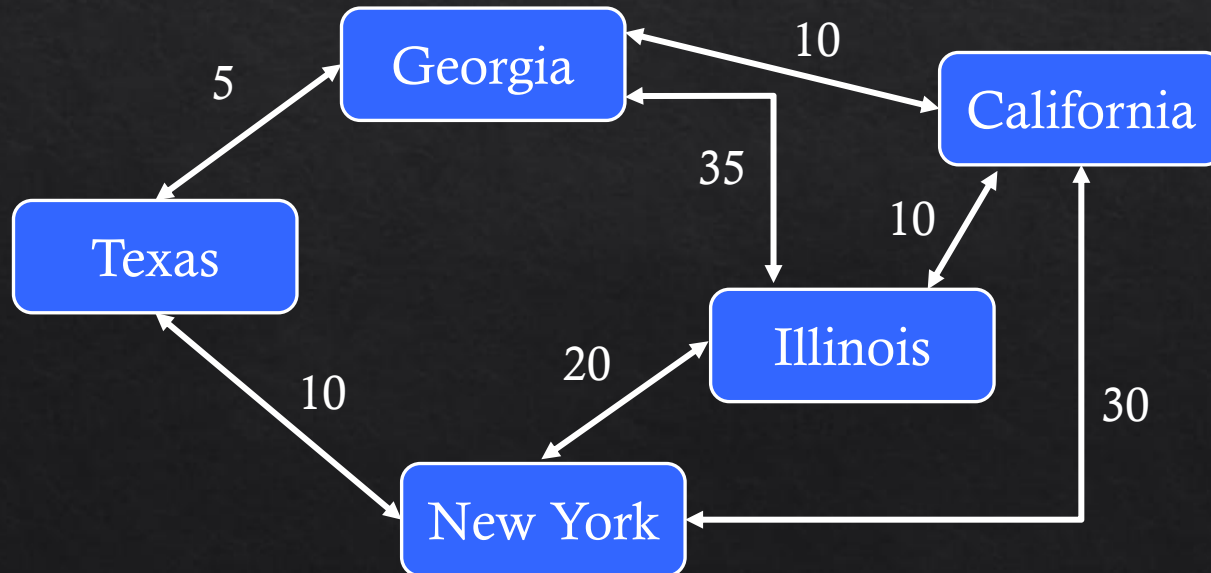
DFS

Single Source Shortest Path Problem

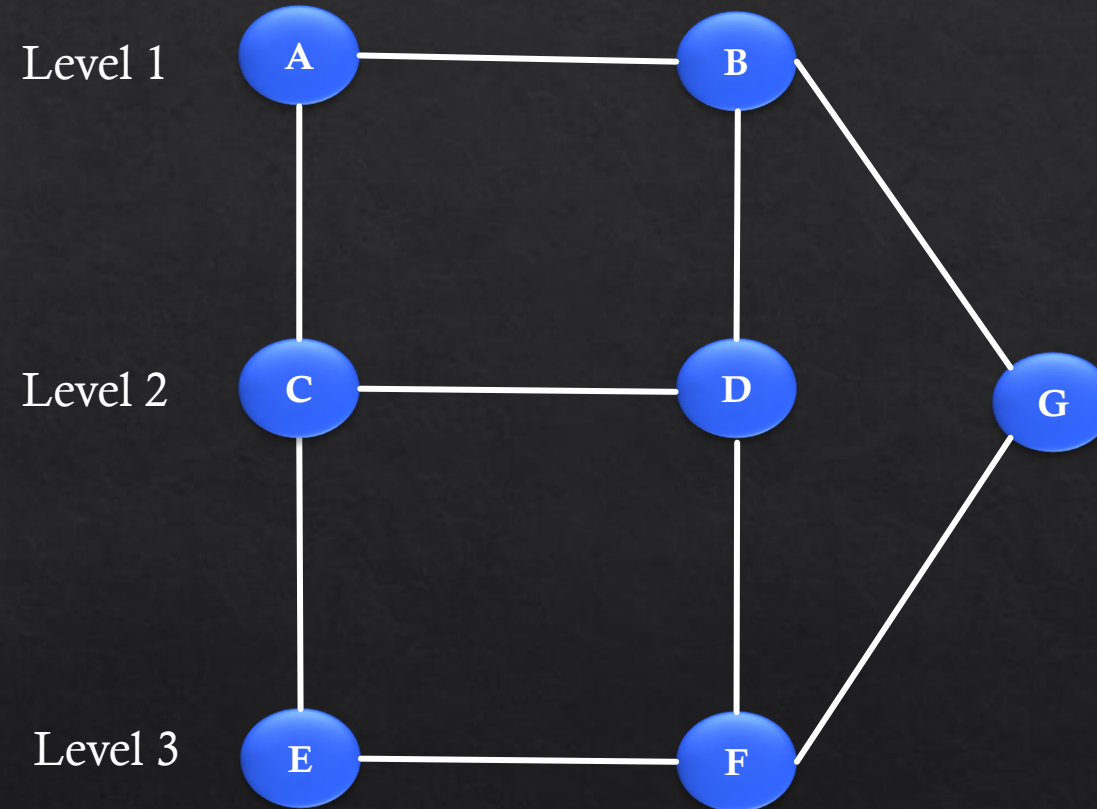
A single source shortest path problem is about finding a path between a given vertex and all other vertices in a graph such that the total distance between them is minimum.

The problem:

- Five offices in different states
- Travel costs between these states are known
- Find the cheapest way from the head office to the branches in different states.



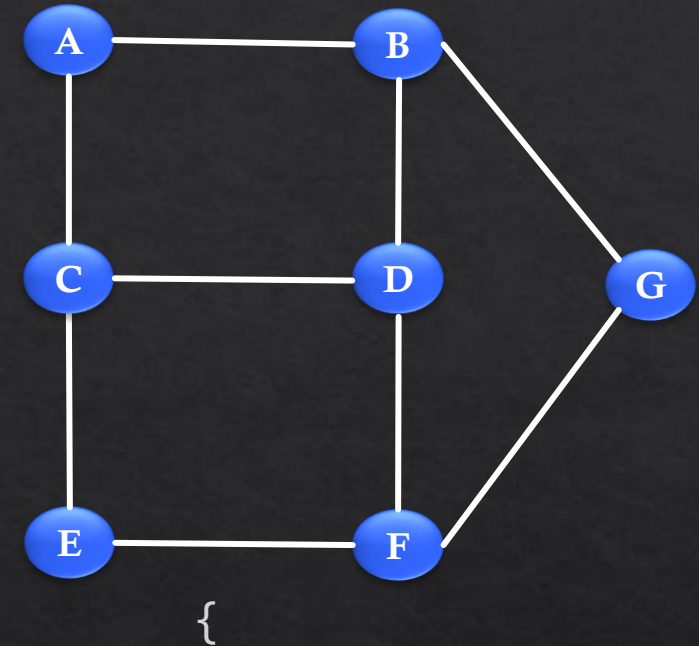
BFS



BFS

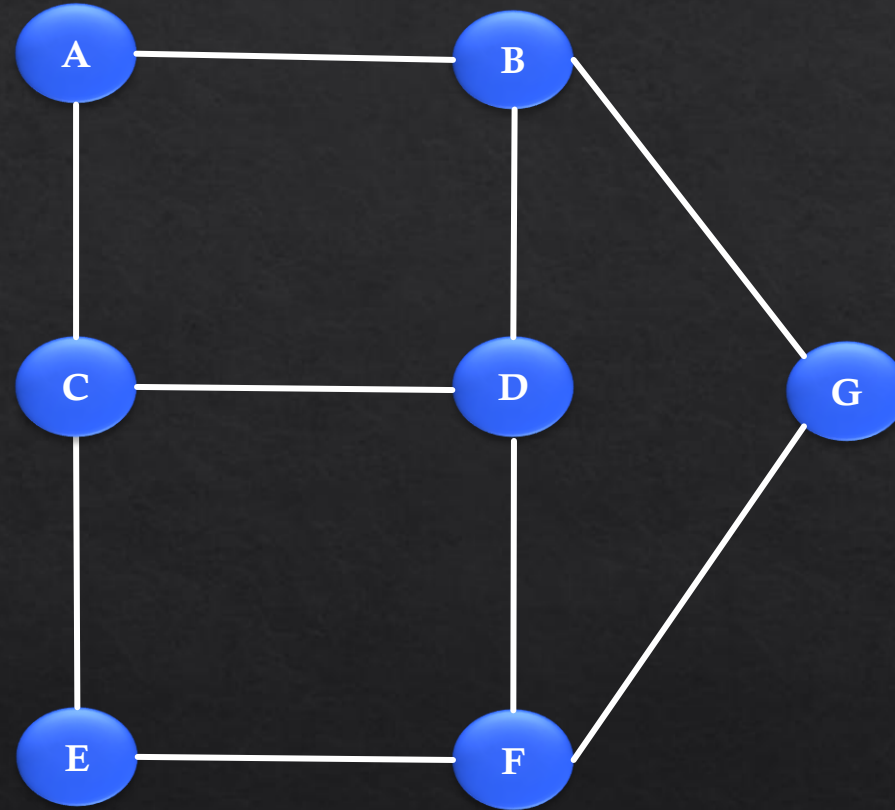
BFS – SSSPP

```
def bfs(self, start, end):  
    queue = []  
    queue.append([start])  
    while queue:  
        path = queue.pop(0)  
        node = path[-1]  
        if node == end:  
            return path  
        for adjacent in self.gdict.get(node, []):  
            new_path = list(path)  
            new_path.append(adjacent)  
            queue.append(new_path)
```



A: [B, C],
B: [A, E],
C: [A, D],
D: [C, E],
E: [B, D]

DFS – SSSPP



DFS